

### 1. What framework are you using to productionize the Data Scientist's model?

We are using **FastAPI** to productionize the model as a RESTful API service. The trained ML model is serialized using **Joblib** and loaded into the backend service. The API runs on **Uvicorn** as the ASGI server, ensuring high performance and asynchronous request handling. This setup enables scalable deployment and seamless integration with web or mobile applications.

### 2. For the user intent classification, what is the exact expected JSON input payload, and what does the output response look like?

```
{
  "query": "predict the sales ",
  "intent": "prediction",
  "confidence": 0.95,
  "intent_scores": {
    "prediction": 1,
    "compare": 0,
    "trend": 0,
    "summarise": 0
  },
  "message": "Forecast for sales_units for next 4 periods",
  "visualization": {
    "type": "area_chart",
    "format": "base64_png",
    "data": "iVBORw0KGgoAAAANSUuEugAABcwAAAN0CAYAAACJBGs1AAAAOnRWFHRTb2Z0d2FyZQBNYXRwbG90bGliIHZlcnNpb24zLjEwLjgsIGh0dHBzOi8vbW0cGxvdGxpYi5vcmlkLThpZWZ0Lj09AAAC"
  },
  "summary": "Cumulative forecast indicates upward growth pattern",
  "full_summary": null,
  "query_params": {
    "target": "sales_units"
  }
}
```

### 3. Are you logging model drift and if so, where?

Yes, we are logging model drift metrics. Input distributions and prediction confidence scores are monitored and stored in structured log files. These logs are maintained using Python logging mechanisms and optionally stored in monitoring dashboards such as **Prometheus** for metric tracking. Drift analysis is performed by comparing current prediction distributions with baseline training distributions.

### 4. How are you handling latency to ensure the pipeline doesn't bottleneck?

Latency is handled through:

- Asynchronous request handling using FastAPI
- Efficient model loading at server startup (avoiding reload per request)
- Lightweight preprocessing pipeline
- Running the service with Uvicorn workers for parallel processing
- Optional caching for repeated requests